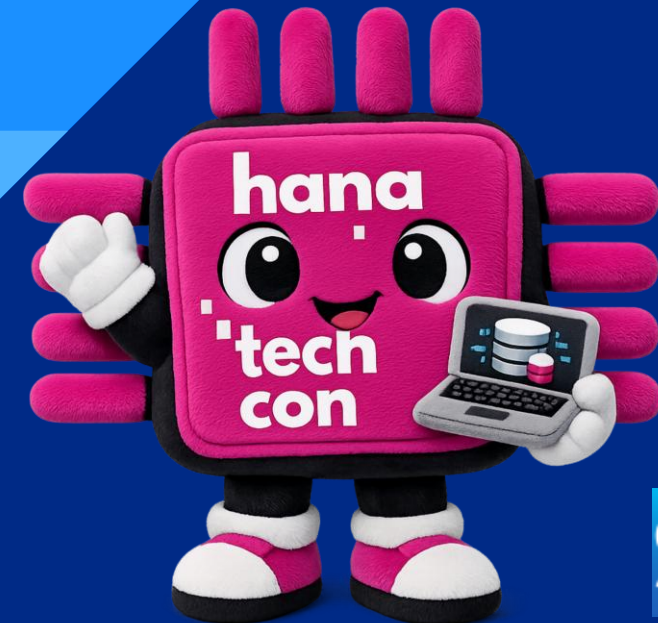


Adaptive Dictionary Scan in HEX: Smarter Column Store Queries in SAP HANA

Oliver Horn, Cheng-Lun Li



Agenda

- Why query optimization continues at runtime
- Column store data structure
- Processing a column store table search in SAP HANA via HEX
- Table Scan - Prefilter vs. Postfilter
- Adaptive Dictionary Scan
- Demo & Result

Why query optimization continues at runtime

Real world data is often does **not** follow a **uniform distribution**

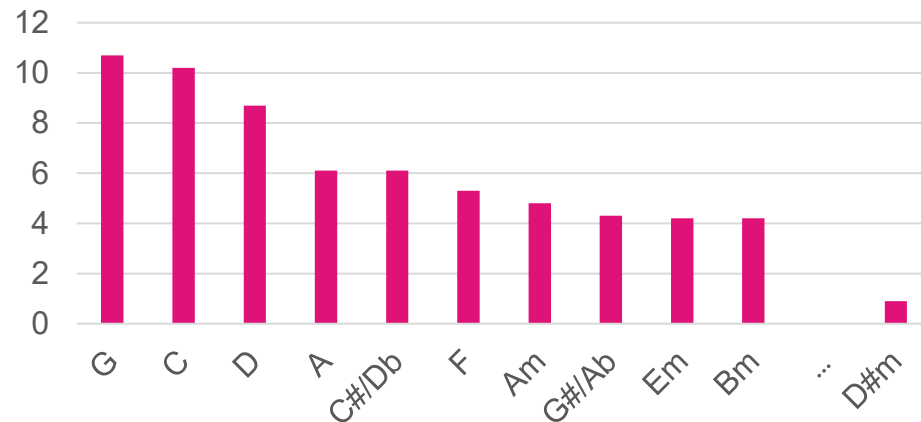
Example 1: Musicians can freely choose from 24 musical keys, yet some keys are dominating

Example 2: Song count vs. their stream count

Imagine to search through a music catalog:

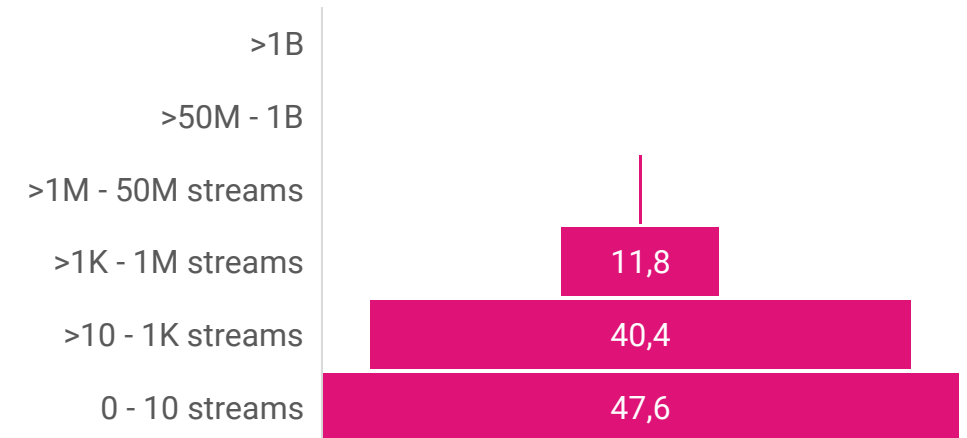
```
SELECT SONG FROM MUSIC_CATALOG WHERE MUSICAL_KEY = ? AND STREAM_COUNT > ?
```

Distribution of keys in music (%)



<https://insights.spotify.com/us/2015/05/06/most-popular-keys-on-spotify/>

Global music streaming pyramid (%)



Luminate Year End Music Report

Why query optimization continues at runtime

Imagine to search through a music catalog:

```
SELECT SONG FROM MUSIC_CATALOG WHERE MUSICAL_KEY = ? AND STREAM_COUNT > ?
```

What order is best to evaluate the filter predicates?

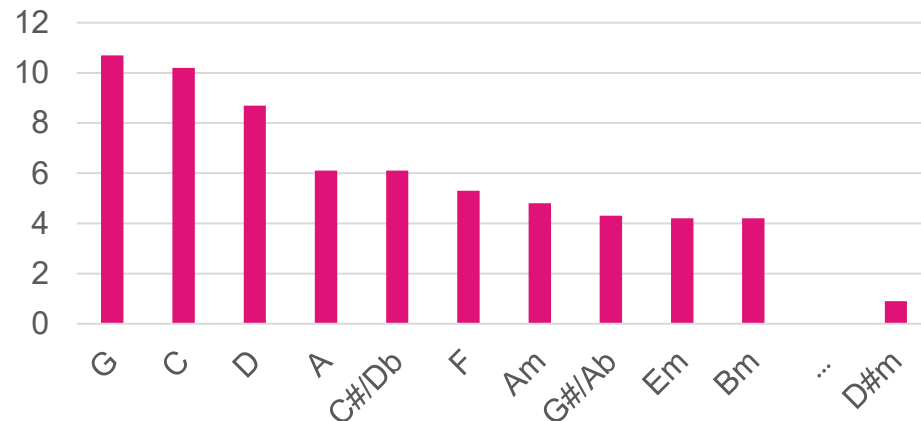
1) `STREAM_COUNT > ?1` 2) `MUSICAL_KEY = ?2`

→ Good if ?1 > 1M (intermediate result < 0.02 %), bad if ?1 ≤ 10 (> 50 %)

2) `MUSICAL_KEY = ?2` 2) `STREAM_COUNT > ?1`

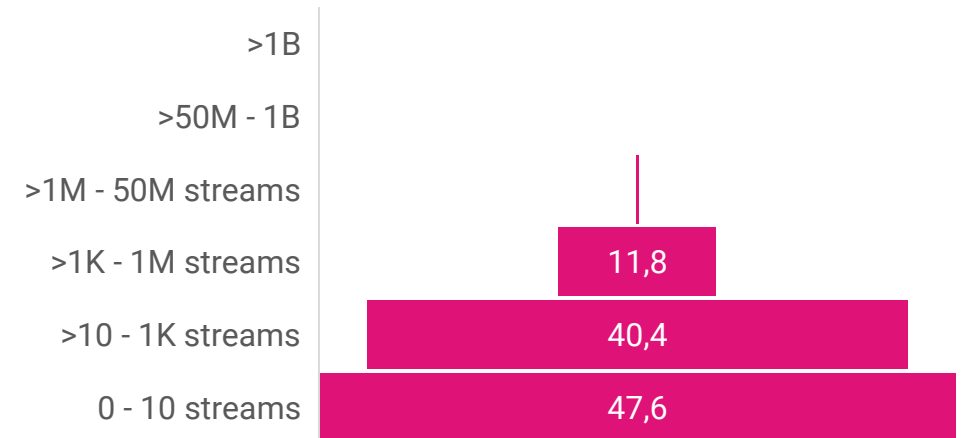
→ Good if ?2 = D#m (< 1 %), bad if ?2 = G/C (~10 %)

Distribution of keys in music (%)



<https://insights.spotify.com/us/2015/05/06/most-popular-keys-on-spotify/>

Global music streaming pyramid (%)



Luminate Year End Music Report

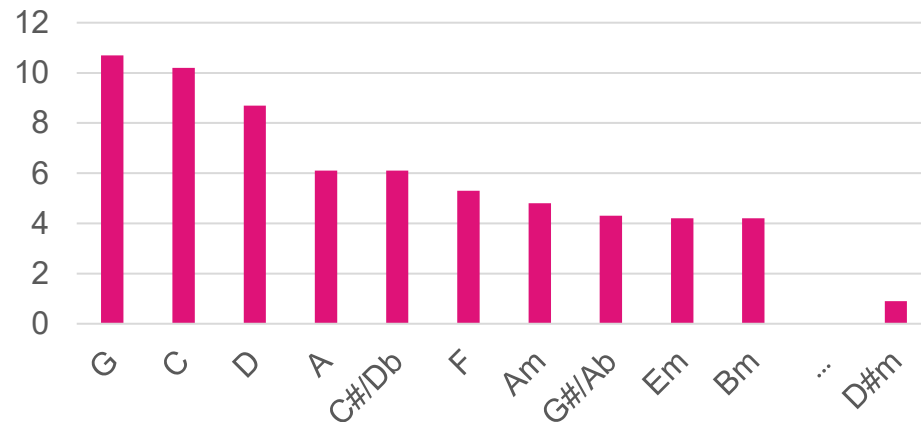
Why query optimization continues at runtime

Imagine to search through a music catalog:

```
SELECT SONG FROM MUSIC_CATALOG WHERE MUSICAL_KEY = ? AND STREAM_COUNT > ?
```

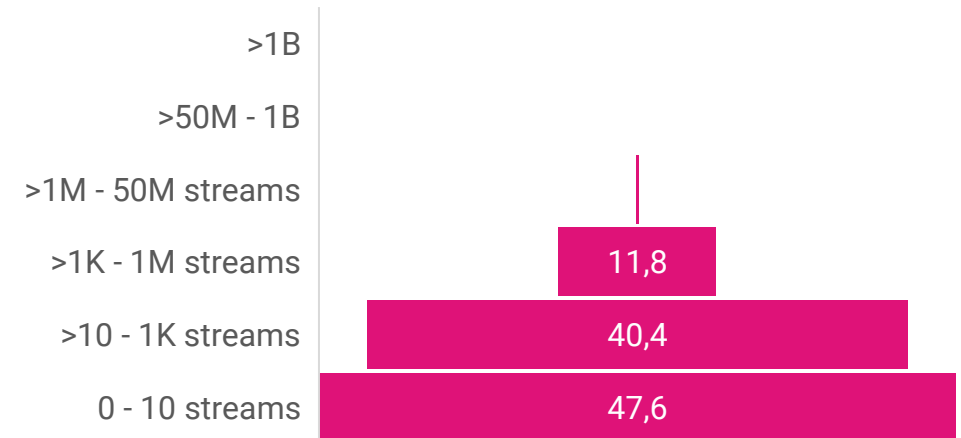
- the dictionary of MUSICAL_KEY is small (24 values) and fast to scan
- the dictionary of STREAM_COUNT is large (> 100M values), would be slow to scan through, thanks to sortedness we can do binary search here

Distribution of keys in music (%)



<https://insights.spotify.com/us/2015/05/06/most-popular-keys-on-spotify/>

Global music streaming pyramid (%)



Luminate Year End Music Report

Why query optimization continues at runtime

Imagine to search through a music catalog:

```
SELECT SONG FROM MUSIC_CATALOG WHERE MUSICAL_KEY = ? AND STREAM_COUNT > ?
```

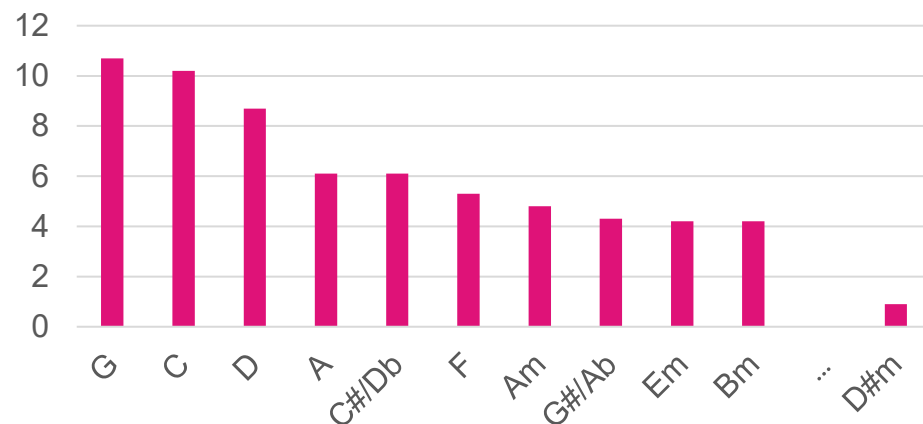
What if we add another filter: `AND ARTIST_NAME LIKE ?`

→ expensive full dictionary scan for arbitrary patterns, e.g. %bob%

→ cheap binary search for prefix patterns such as Bob%

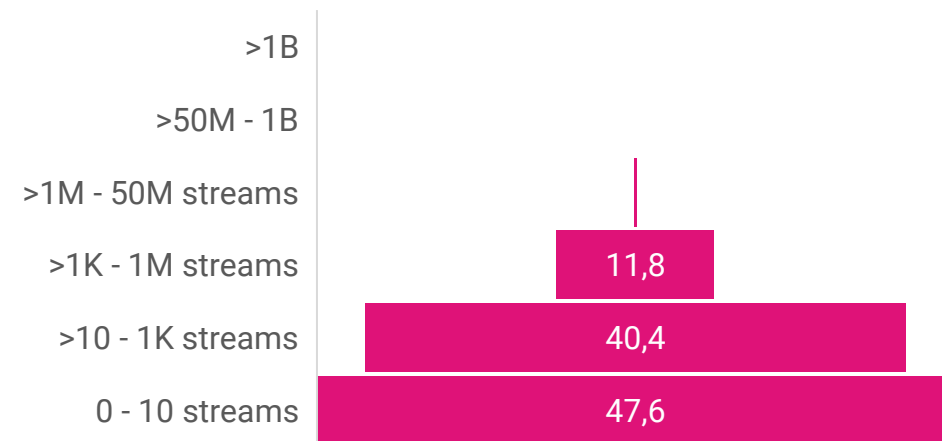
→ if the first two predicates only match few rows, we can instead evaluate only the few rows' values

Distribution of keys in music (%)



<https://insights.spotify.com/us/2015/05/06/most-popular-keys-on-spotify/>

Global music streaming pyramid (%)



Luminate Year End Music Report

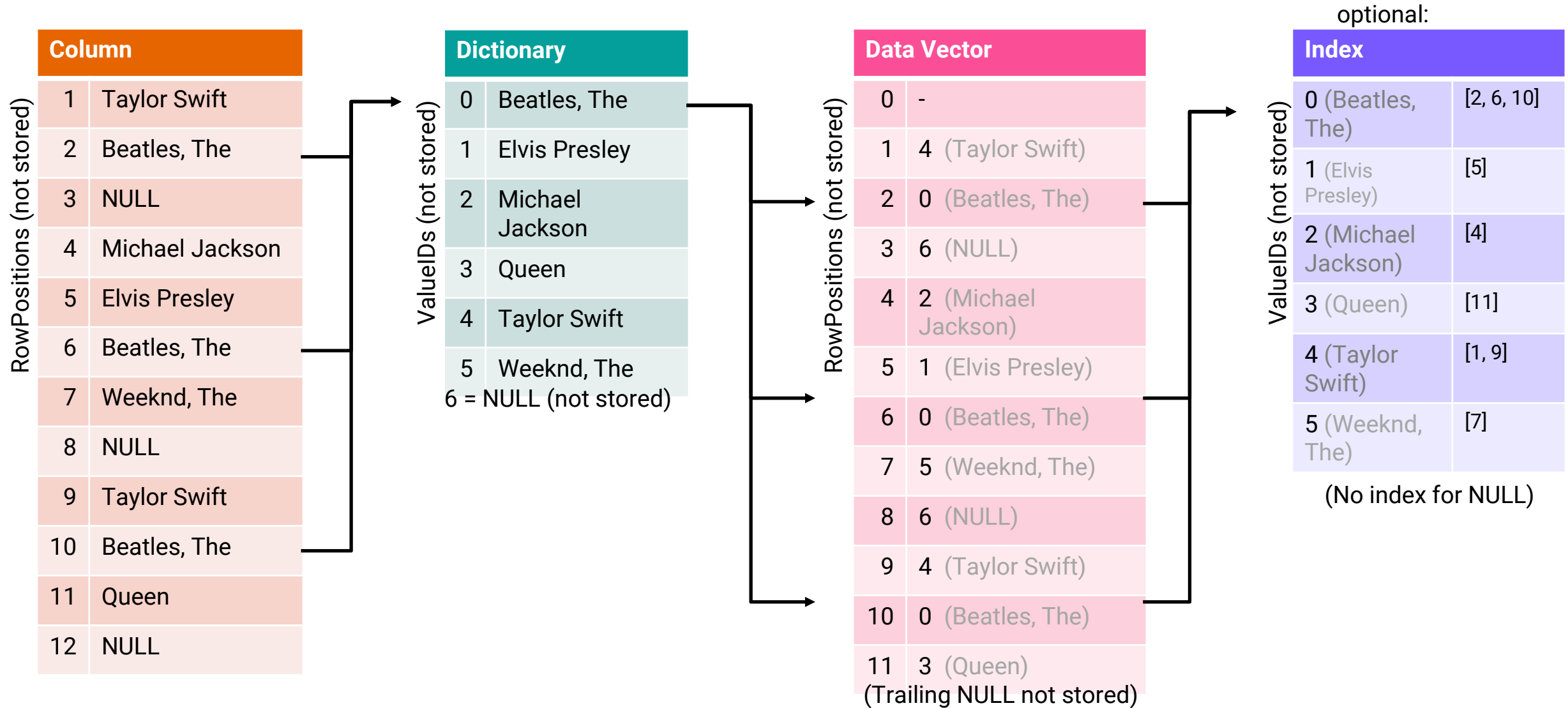
Why query optimization continues at runtime

Many evaluation paths are possible

Compiling a plan with fixed order and evaluation strategy for a parameterized query can not be optimal for all cases.

→ We need flexibility in the query plan to adapt at runtime to the given query parameters.

Column store data structure (Main)



Processing a column store table search in SAP HANA via HEX

1. Table Scan

- Most generic way to evaluate a column store table with/without WHERE clause
- 3 steps (pipelines) for evaluating column store structures: dictionary, index, data vector
- Produce filtered result set

2. Index Scan

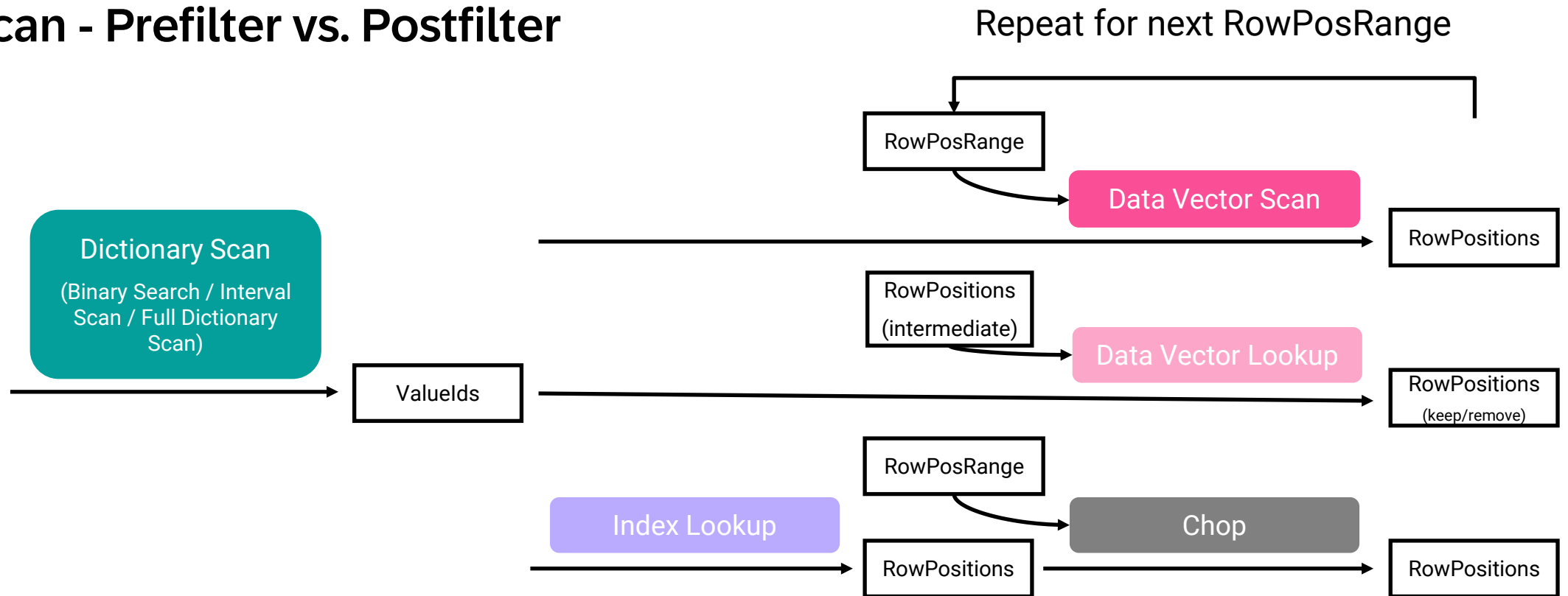
- Fast evaluation of MIN/ MAX / (TOP) ORDER BY queries leveraging an available index
- Produce sorted result set, then apply filter

3. Unique Index Lookup

- Fast single row access (primary key lookup / other unique index)

Table Scan - Prefilter vs. Postfilter

Prefilter



Postfilter

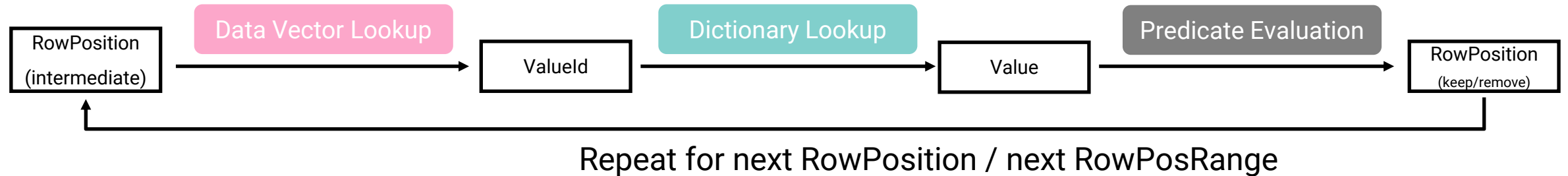


Table Scan - Prefilter vs. Postfilter

Prefilter

- Highly optimized and specialized for column store
- Uses pre-compiled code
- Various optimizations: binary search, full dictionary scan, index lookup, ...

Postfilter

- Generic way to evaluate predicate row by row
- Uses generated code
- Faster if the intermediate rows are small

But...

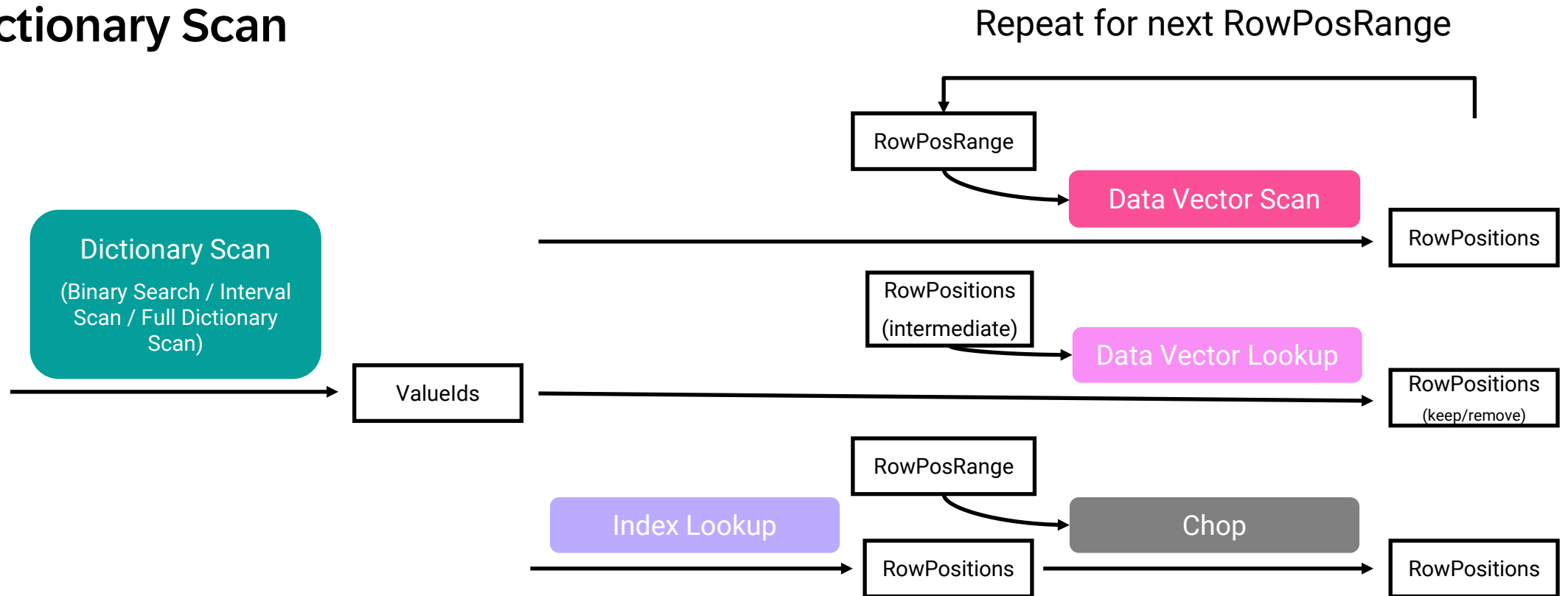
- Dictionary scan can be **expensive if dictionary is large**
- Dictionary scan can be expensive for some filters, for example **LIKE '%bob%'**.
- In a conjunctive filter, we can leverage the small result from previous filter (like in postfilter)
- Prefilter/Postfilter decision is done during compile time with limited information available.
 - **LIKE 'Bob%'** vs. **LIKE '%bob%'**
 - Dictionary size
 - Result from previous filter during runtime
- Prefilter/Postfilter **not runtime adjustable**

Adaptive Dictionary Scan

- **Early Dictionary Scan:** Dictionary Scan done as just described
- **Late Dictionary Scan:** Post-filter like evaluation when RowPosition is available
- **Adaptive Dictionary Scan:** Decide at runtime to do Early or Late Dictionary Scan
- Config
 - [hex] dict_scan_mode (early/adaptive/late)
- Hint
 - HEX_TABLE_SCAN_EARLY_DICT_SCAN
 - HEX_TABLE_SCAN_ADAPTIVE_DICT_SCAN
 - HEX_TABLE_SCAN_LATE_DICT_SCAN

Early Dictionary Scan

Prefilter

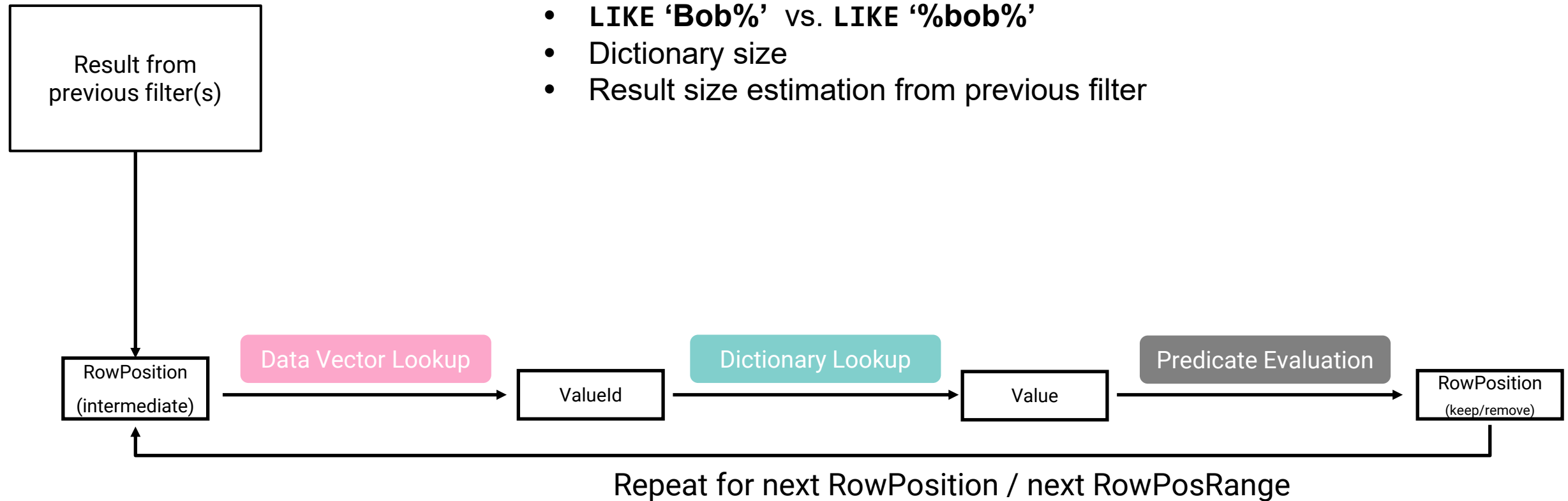


Late Dictionary Scan

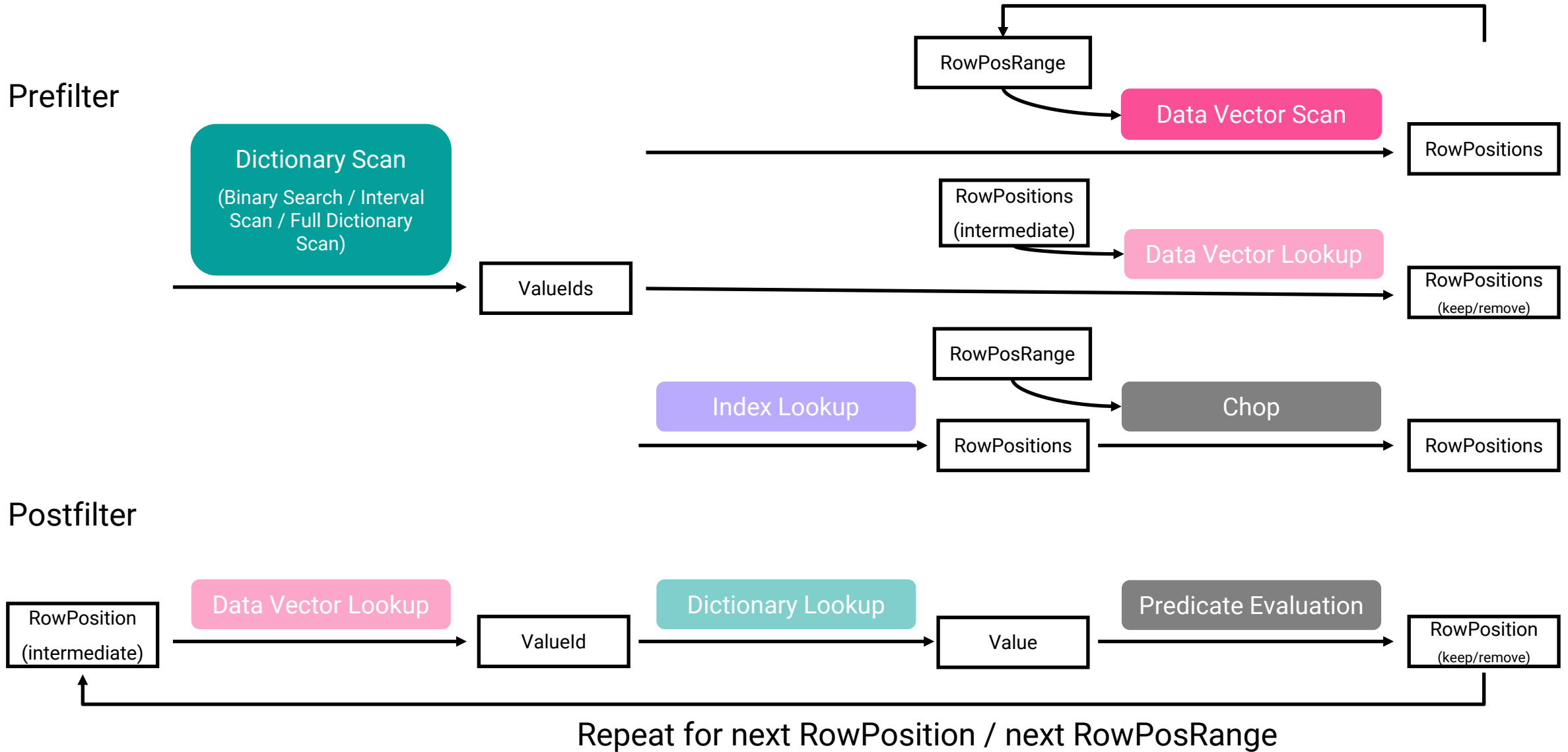
Prefilter

Differences with Postfilter

- Postfilter is codegen. Unchangable at runtime
- Adaptive DictScan reacts according to **runtime metrics**.
 - **LIKE 'Bob%'** vs. **LIKE '%bob%'**
 - Dictionary size
 - Result size estimation from previous filter



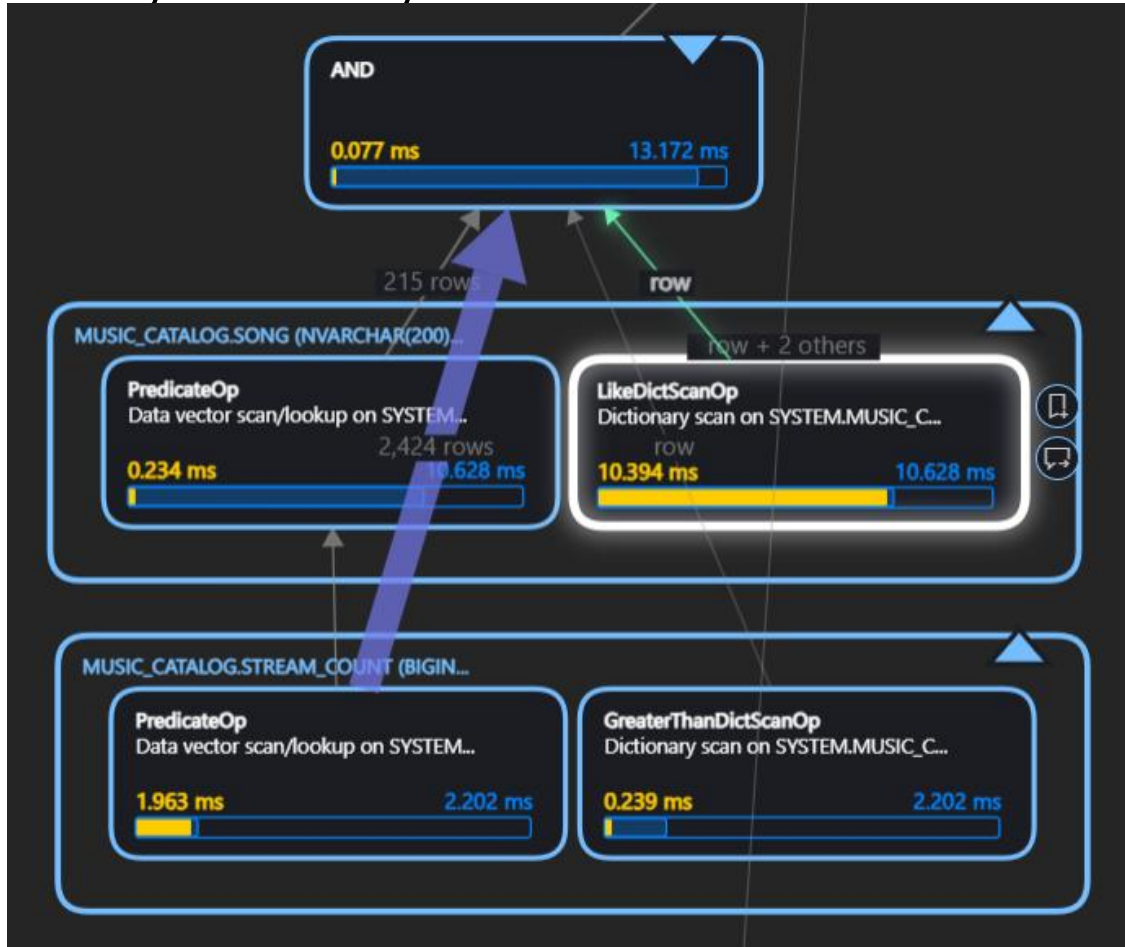
Prefilter vs. Postfilter (recap)



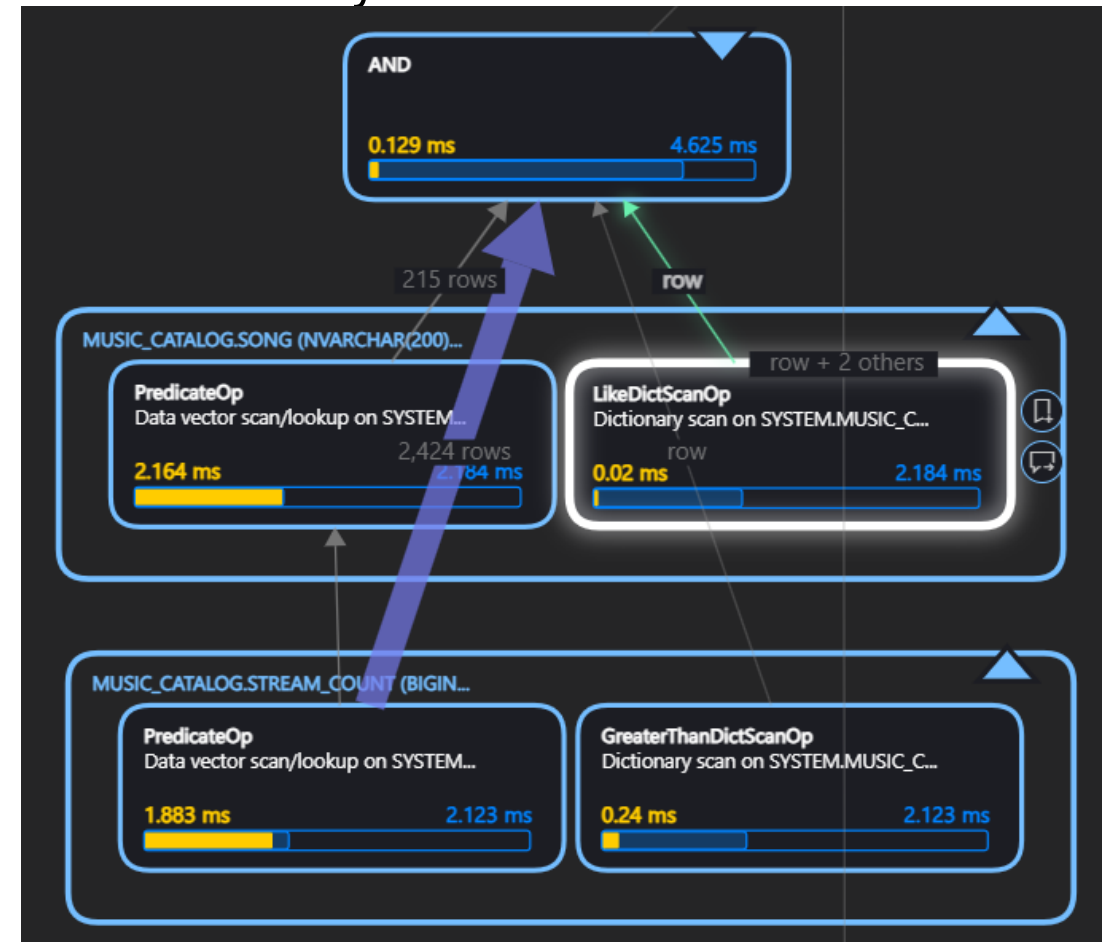
Result

SELECT * from MUSIC_CATALOG WHERE
STREAM_COUNT > ? AND SONG LIKE ?

Early Dictionary Scan



Late Dictionary Scan



Result: up to 99.9% improvement

	RATIOAVG 	RATIOBST	Less is better	Config	Test	Series	Category
	99.94%	99.94%	T	MainPL_WithPK	P500	FULL_NUMC_GREATER_U	elapsed time avg
	99.94%	99.94%	T	MainPL_WithPK	P2000	FULL_NUMC_GREATER_U	elapsed time avg
	99.94%	99.94%	T	MainPL_WithPK	P500	FULL_STRING_GREATER_U	elapsed time avg
	99.94%	99.94%	T	MainPL_NoPK	P2000	FULL_NUMC_GREATER_U	elapsed time avg
	99.94%	99.94%	T	MainPL_NoPK	P2000	FULL_STRING_GREATER_U	elapsed time avg
	99.94%	99.94%	T	MainPL_WithPK	P2000	FULL_STRING_GREATER_U	elapsed time avg
	99.85%	99.86%	T	Main_WithPK	P500	FULL_STRING_GREATER_U	elapsed time avg
	99.84%	99.85%	T	Delta_WithPK	P500	FULL_STRING_GREATER_U	elapsed time avg
	99.84%	99.84%	T	Main_WithPK	P500	FULL_NUMC_GREATER_U	elapsed time avg
	99.83%	99.84%	T	Delta_WithPK	P500	FULL_NUMC_GREATER_U	elapsed time avg
	99.83%	99.83%	T	Main_WithPK	P2000	FULL_STRING_GREATER_U	elapsed time avg
	99.83%	99.83%	T	Main_NoPK	P2000	FULL_STRING_GREATER_U	elapsed time avg
	99.82%	99.82%	T	Delta_WithPK	P2000	FULL_STRING_GREATER_U	elapsed time avg
	99.82%	99.82%	T	MainPL_WithPK	P500	TENTH_STRING_LIKE	cpu-service-time-avg

Thank you.

Contact information:

Oliver Horn
oliver.horn@sap.com

Cheng-Lun Li
cheng-lun.li@sap.com